

École Polytechnique de Sousse

Année universitaire 2026–2027

Cours MERN Stack – Séance 1

Compte rendu

Mon premier serveur Express : l'API d'un blog

Étudiante : Rouatbi Hedil

Classe : 5^e DS, Groupe 3

Enseignant : Dr. Abdelweheb Gueddes

Dépôt Git : https://github.com/hedil1/TP1_MERN-Stack

Table des matières

1	TP guidé : l'API du blog	2
2	Exercice 1 : compléter l'API	4
2.1	GET /about	4
2.2	GET /api/users	5
2.3	GET /api/users/:id	6
2.4	POST /contact	7
2.5	Bonus : GET /api/users?name=Aya	8
3	Exercice 2 : les notions JavaScript	9
4	Dépôt Git	10

1. TP guidé : l'API du blog

Étape 2 : un serveur qui dit bonjour

```
1  const express = require('express'); // 1. charger la bibliothèque Express
2  const app = express();               // 2. créer l'application : c'est notre serveur
3  const PORT = 3000;
4  app.use(express.json());
5
6  // 3. une route : quand un client demande GET /, Express exécute cette fonction
7  app.get('/', (req, res) => {
8    res.json({ message: "Bonjour, je suis l'API du blog" });
9  });
```

Exécution : on lance le serveur, puis on ouvre `http://localhost:3000` dans le navigateur.

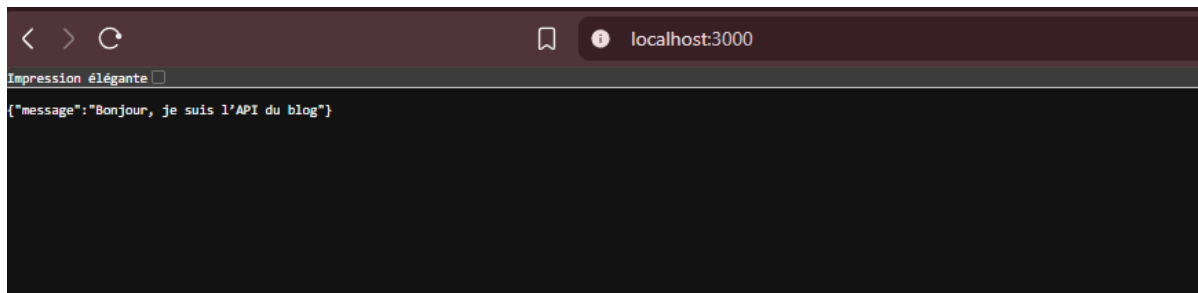


Figure 1: Réponse de la route GET / dans le navigateur

Étape 3 : la liste des articles

```
1  const articles = [
2    { id: 1, title: 'Bienvenue sur le blog', author: 'Admin' },
3    { id: 2, title: 'Mon premier serveur Express', author: 'Aya' },
4    { id: 3, title: 'Tester une API avec Postman', author: 'Aya' }
5  ];
6
7  // GET /api/articles -> tous les articles
8  app.get('/api/articles', (req, res) => {
9    res.json({ total: articles.length, articles: articles });
10 });
```

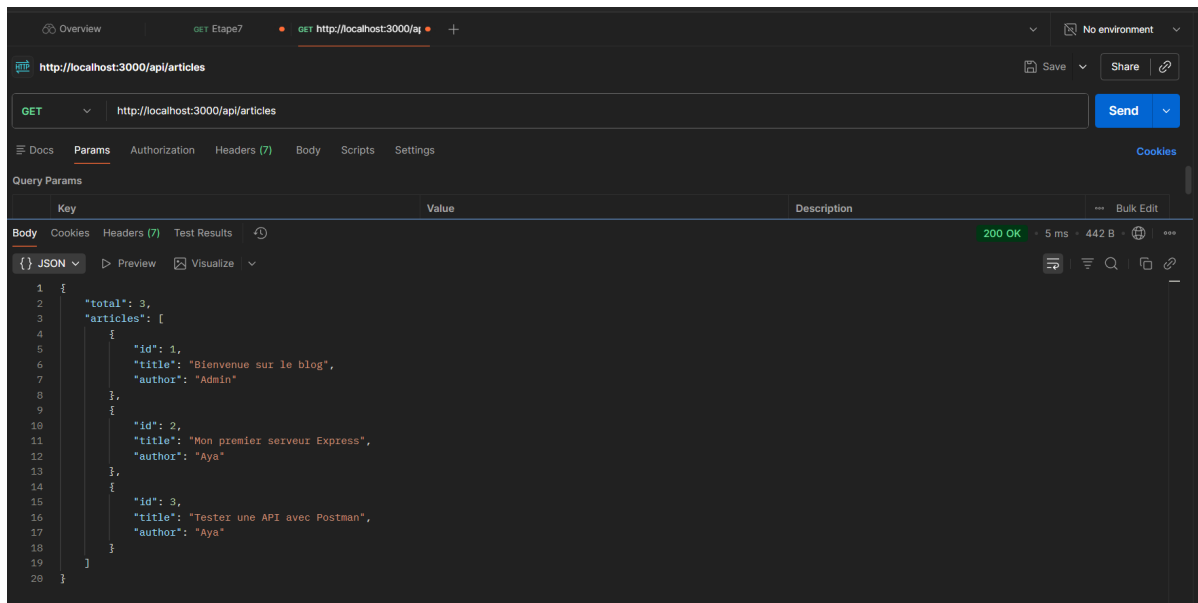


Figure 2: Test Postman : GET /api/articles (200 OK, 3 articles)

Étape 4 : un seul article

```

1 // GET /api/articles/2 -> l'article dont l'id vaut 2
2 app.get('/api/articles/:id', (req, res) => {
3   const id = Number(req.params.id); // "2" -> 2
4   const article = articles.find(a => a.id === id);
5
6   if (!article) {
7     return res.status(404).json({ error: `Article ${id} introuvable` });
8   }
9
10  res.json(article);
11 });

```

L'identifiant lu dans l'URL est toujours du *texte* : `req.params.id` vaut "2". Comme "2" === 2 est faux, il faut le convertir avec `Number()` avant de le comparer.

Étape 5 : filtrer par auteur

```

1 // GET /api/articles?author=Aya -> seulement ceux d'Aya
2 app.get('/api/articles', (req, res) => {
3   const { author } = req.query; // = const author = req.query.author;
4   let resultat = articles;
5
6   if (author) { // si le client a précisé ?author=...
7     resultat = articles.filter(a => a.author === author);
8   }
9
10  res.json({ total: resultat.length, articles: resultat });
11 });

```

Étape 6 : créer un article avec POST

```

1 let prochainId = 4; // le prochain id à attribuer (les ids 1, 2 et 3 existent déjà)
2

```

```

3 // POST /api/articles -> crée un article avec { "title": "...", "author": "..." }
4 app.post('/api/articles', (req, res) => {
5   const { title, author } = req.body; // déstructuration
6
7   if (!title || !author) { // validation : les deux champs sont obligatoires
8     return res.status(400).json({ error: "Le titre et l'auteur sont obligatoires" });
9   }
10
11   const nouvelArticle = { id: prochainId, title: title, author: author };
12   prochainId = prochainId + 1;
13
14   articles.push(nouvelArticle);
15
16   res.status(201).json({ message: 'Article créé', article: nouvelArticle });
17 });

```

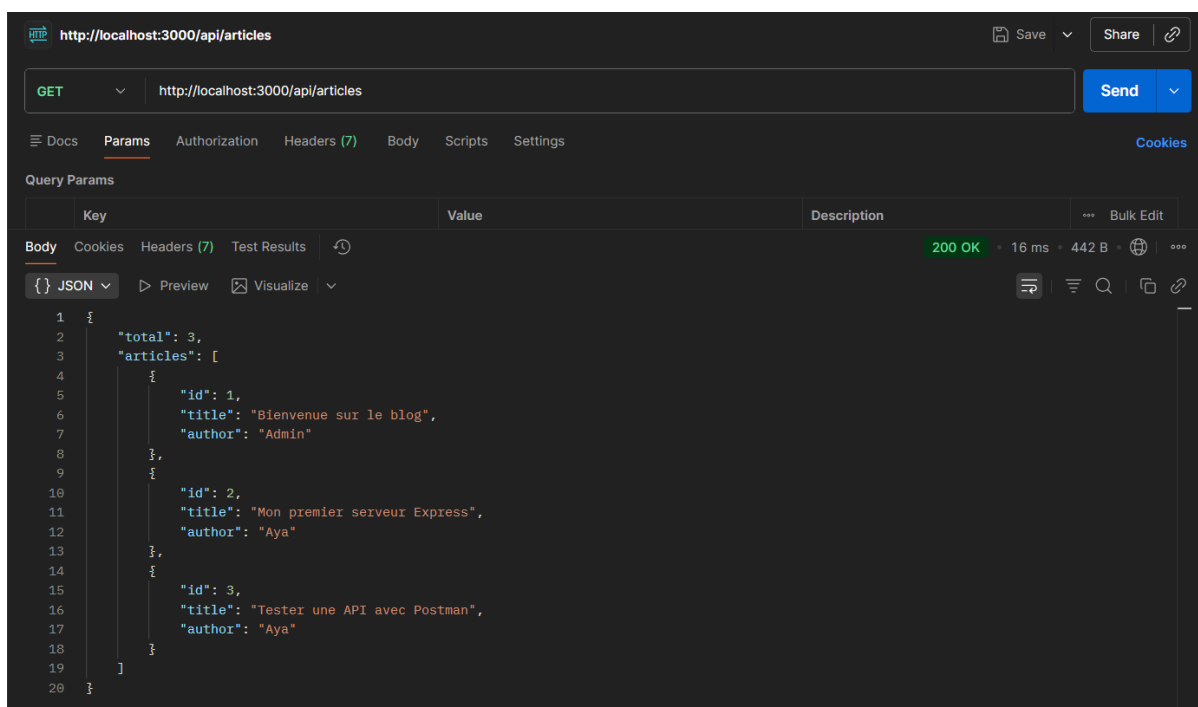


Figure 3: Test Postman : GET /api/articles

2. Exercice 1 : compléter l'API

2.1 GET /about

Renvoie un objet avec le nom de l'application, le nom de l'auteur et la version.

```

1 app.get('/about', (req, res) => {
2   res.json({
3     application: 'API du blog',
4     auteur: 'Votre Nom',
5     version: '1.0.0'
6   });
7 });

```

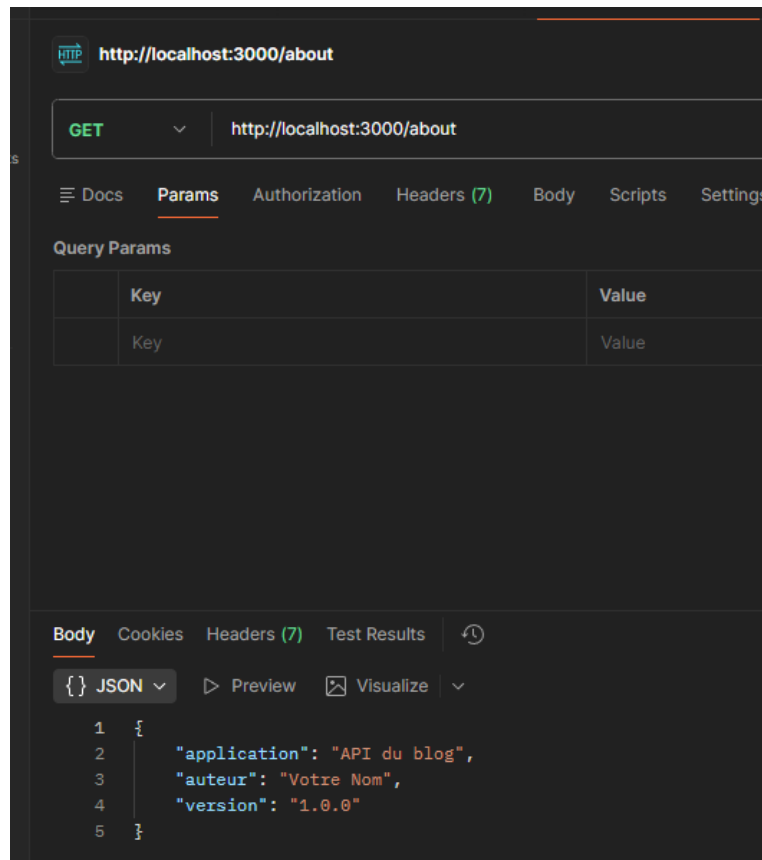


Figure 4: Test Postman : GET /about (200 OK)

2.2 GET /api/users

Renvoie un tableau de trois utilisateurs (id, name, email) rangé dans une constante users.

```

1  const users = [
2    { id: 1, name: 'Aya', email: 'aya@example.com' },
3    { id: 2, name: 'Omar', email: 'omar@example.com' },
4    { id: 3, name: 'Youssef', email: 'youssef@example.com' }
5  ];
6
7  app.get('/api/users', (req, res) => {
8    const { name } = req.query;
9    let resultat = users;
10   if (name) {
11     resultat = users.filter(u => u.name === name);
12   }
13   res.json({ total: resultat.length, users: resultat });
14 });

```

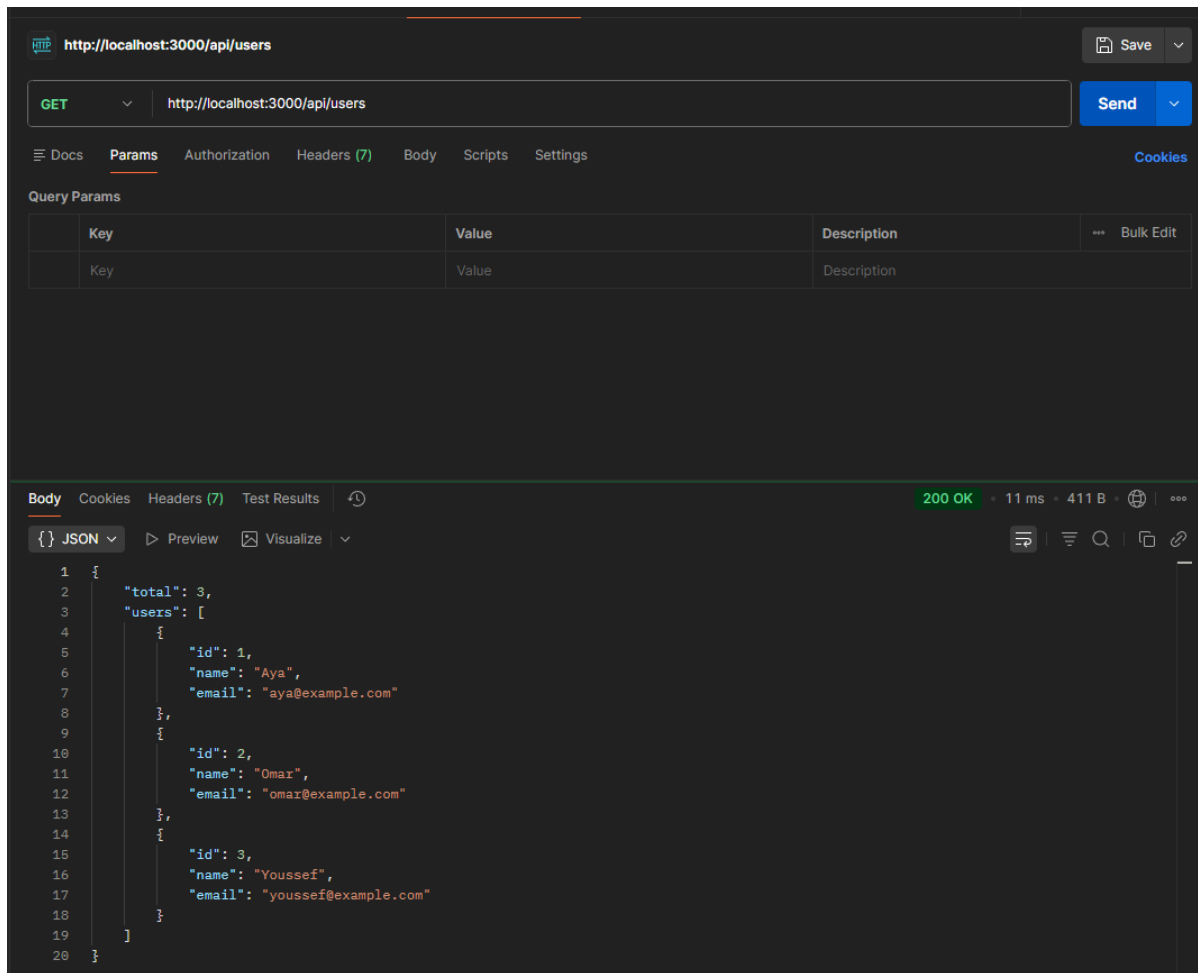


Figure 5: Test Postman : GET /api/users (200 OK, 3 utilisateurs)

2.3 GET /api/users/:id

Renvoie l'utilisateur demandé, ou une erreur 404 s'il n'existe pas (sur le modèle de l'étape 4).

```

1 app.get('/api/users/:id', (req, res) => {
2   const id = Number(req.params.id);
3   const user = users.find(u => u.id === id);
4   if (!user) {
5     return res.status(404).json({ error: `Utilisateur ${id} introuvable` });
6   }
7   res.json(user);
8 });

```

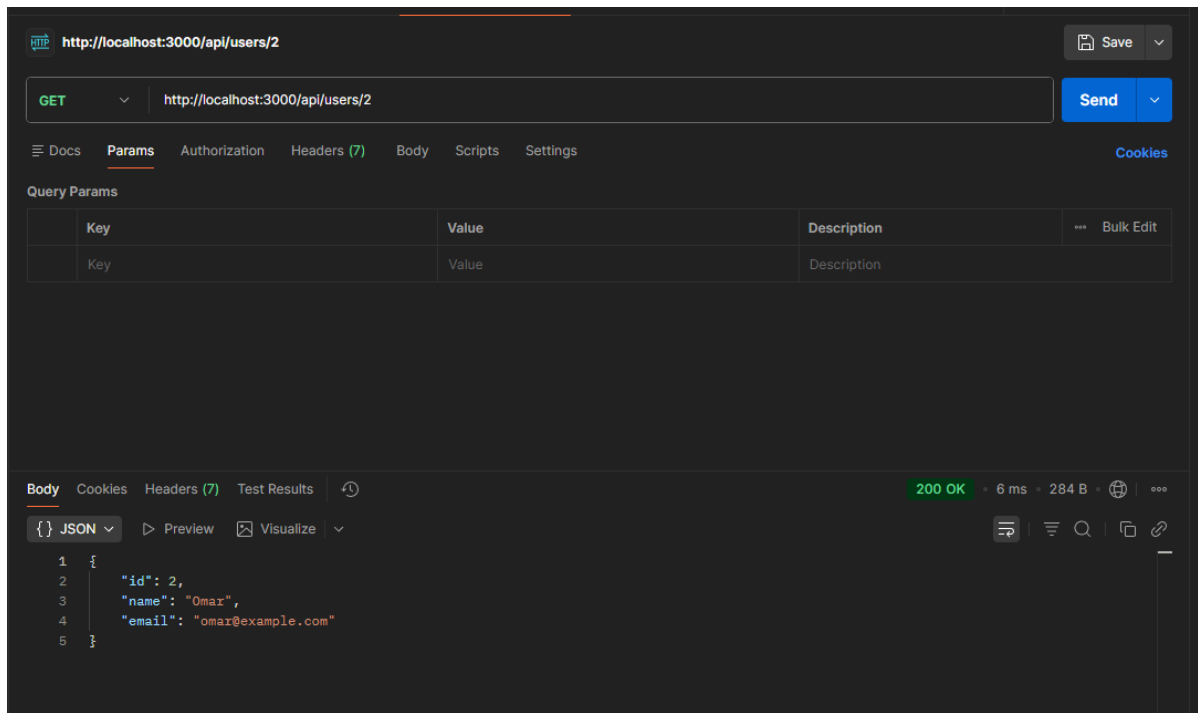


Figure 6: Cas de succès : GET /api/users/2 (200 OK)

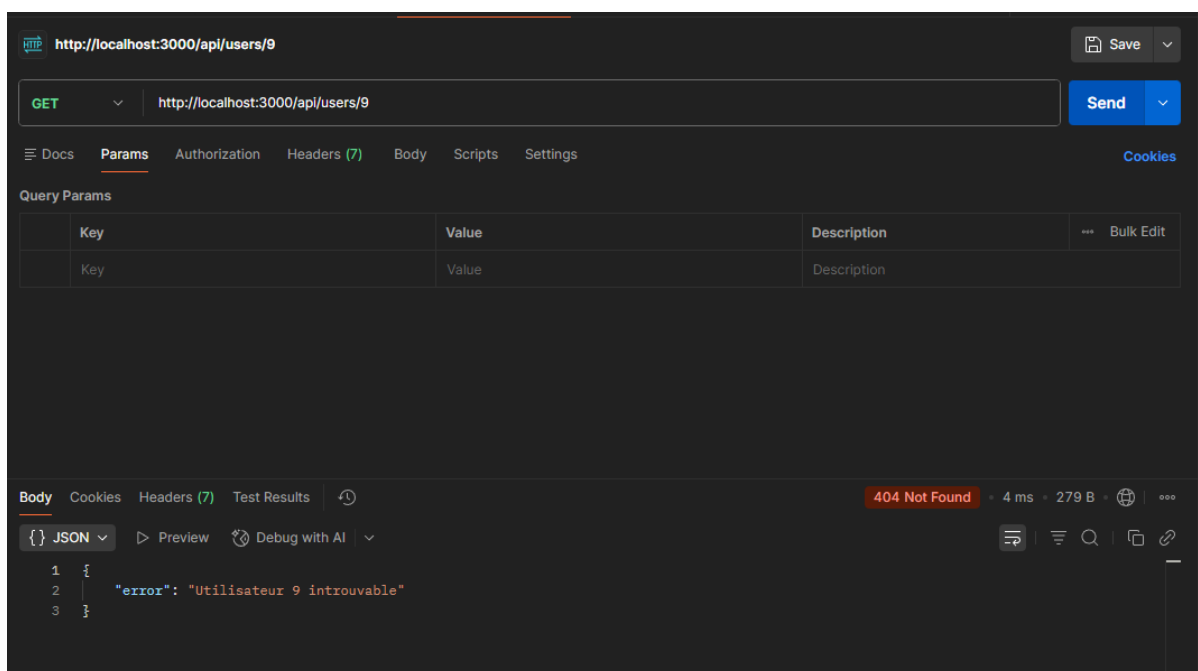


Figure 7: Cas d'erreur : GET /api/users/9 (404 Not Found)

2.4 POST /contact

Reçoit `{ "email": "...", "message": "..." }`. Si un champ manque, la route répond 400 ; sinon elle répond 200 avec `{ "message": "Merci, votre message a bien été reçu" }`.

```
1 app.post('/contact', (req, res) => {
2   const { email, message } = req.body;
3   if (!email || !message) {
4     return res.status(400).json({ error: "L'email et le message sont obligatoires" });
5   }
6 }
```

```

6   res.status(200).json({ message: 'Merci, votre message a bien été reçu' });
7 });

```

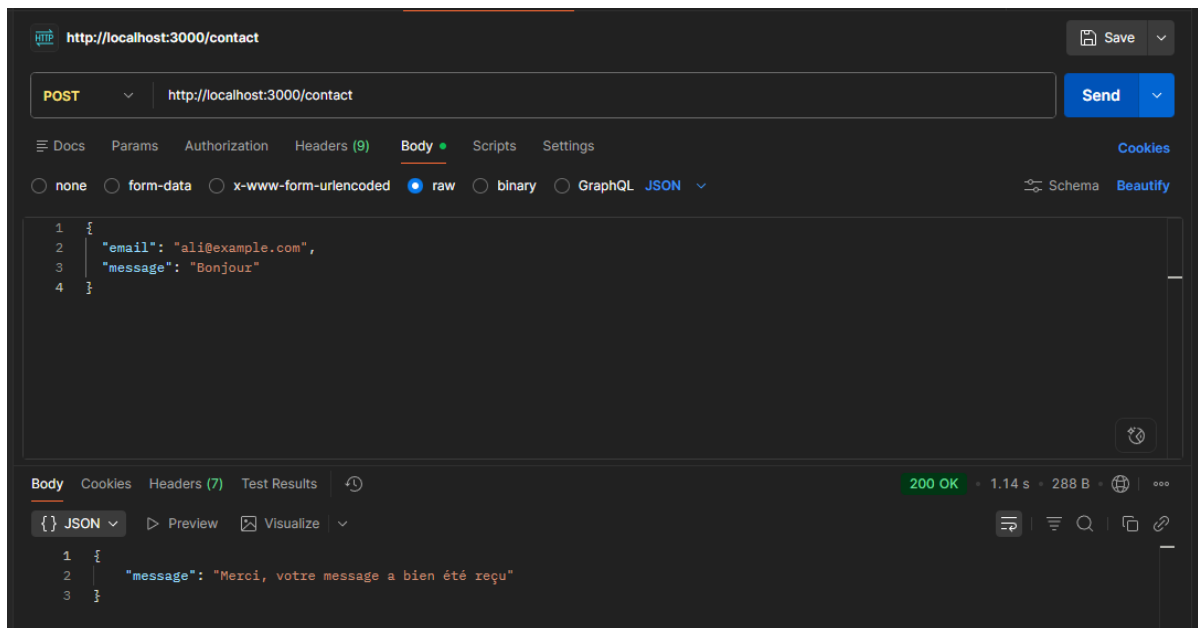


Figure 8: Cas de succès : POST /contact avec email et message (200 OK)

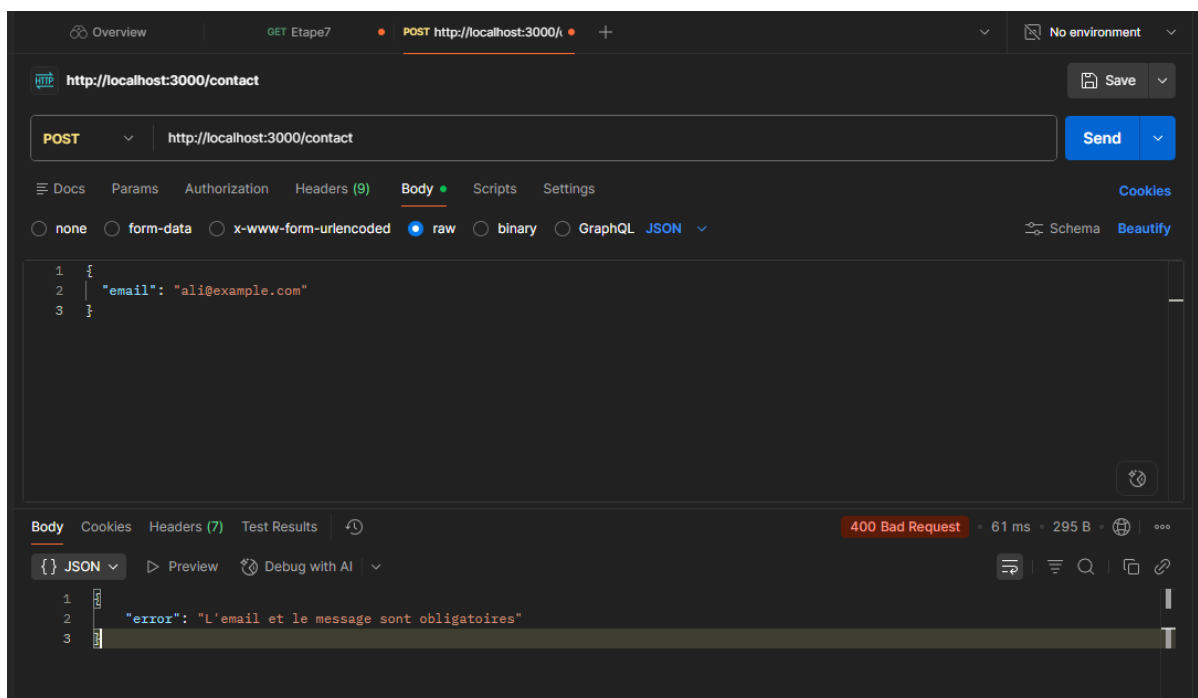


Figure 9: Cas d'erreur : POST /contact sans message (400 Bad Request)

2.5 Bonus : GET /api/users?name=Aya

Filtre les utilisateurs par nom (sur le modèle de l'étape 5). Le filtre est intégré dans la route GET /api/users présentée en section 2.2.

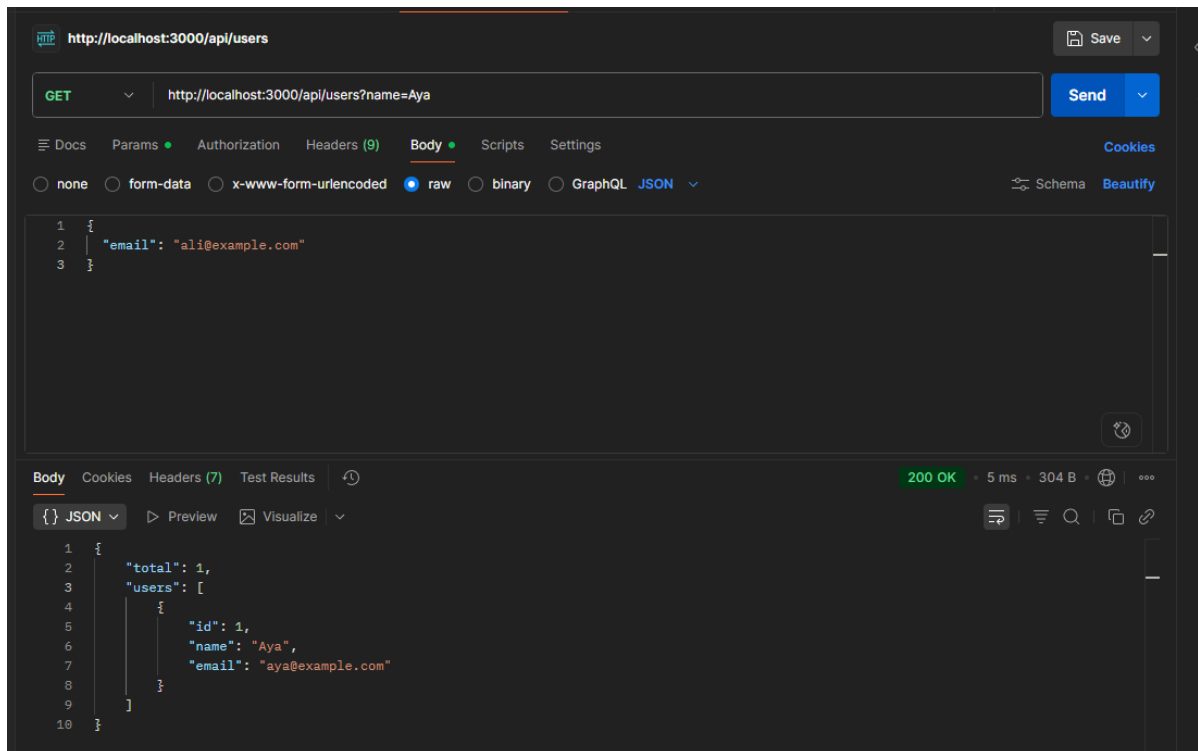


Figure 10: Test Postman : GET /api/users?name=Aya (200 OK, 1 utilisateur)

3. Exercice 2 : les notions JavaScript

```

1  const produits = [
2    { nom: 'Clavier', prix: 45 },
3    { nom: 'Écran', prix: 320 },
4    { nom: 'Souris', prix: 25 }
5  ];
6
7  // Déstructuration
8  const { nom, prix } = produits[0];
9  console.log(nom, prix);
10
11 // produit : Souris
12 const souris = produits.find(p => p.nom === 'Souris');
13 console.log(souris.prix);
14
15 // produits dont le prix est inférieur à 100
16 const pasChers = produits.filter(p => p.prix < 100);
17 console.log(pasChers);
18
19 // fonction fléchée : prix moins 10 %
20 const avecRemise = (prix) => prix * 0.9;
21 console.log(avecRemise(320));
22
23 // transformation de chaque produit
24 const noms = produits.map(p => p.nom);
25 console.log(noms);

```

Sortie dans le terminal (node exercices_js.js):

```

Clavier 45
25
[ { nom: 'Clavier', prix: 45 }, { nom: 'Souris', prix: 25 } ]

```

```
288  
[ 'Clavier', 'Écran', 'Souris' ]
```

Détails

1. **Déstructuration** : `const { nom, prix } = produits[0]` crée deux variables à partir du premier objet.
2. **find** : renvoie le premier objet dont le nom est « Souris » ; on affiche sa propriété `prix` (25).
3. **filter** : renvoie un nouveau tableau avec les produits à moins de 100 (Clavier et Souris).
4. **Fonction fléchée** : `prix * 0.9` retire 10 % : $320 \times 0,9 = 288$.
5. **map** : transforme chaque produit en son nom : on obtient un tableau de même longueur.

4. Dépôt Git

Le code complet du TP (`server.js`, `exercices_js.js`, `package.json`) est disponible sur GitHub :

https://github.com/hedil1/TP1_MERN-Stack